ML Report 1

Ferdinand Chiu, Simon Streuli

October 2026

1 Data acquisition and cleaning

We used one of the helper functions to acquire the data. Since the data have many missing values, we were unable to discard every faulty row. Instead, we had to replace each `NaN` with the median value of each column.

The y training data takes the values -1 or +1. We map -1 to 0 for mathematical consistency with the methods described in the next section.

2 Methods

We use 5-fold cross-validation (seed 12) for regularized logistic regression (300 iterations, learning rate $\eta = 0.02$, $\lambda = 0.001$) before fitting the model on the full training set. Since we use regularized logistic regression, we standardized the features before fitting, to prevent some features getting disproportionally punished. The script reports validation accuracy for each fold and their mean.

Missing feature values are imputed using medians calculated from each fold's training split. For the final fit, medians are calculated from the full training set. To find our weights w, we use regularized logistic regression that you can find in `implementations.py`, although we tried mean square gradient descent and logistic regression during debugging. The code is adapted from the notebooks.

To make our predictions, we simply computed $sign(\sigma(x^\intercal.w))$ to get an array of values in (-1, +1), corresponding to binary decisions.

The code for putting the predictions in a .csv file is from `helpers.py`.

3 Finding optimal parameters and debugging

At first, the mean square gradient descent, the logistic regression, and the regularized logistic regression would all produce errors when running. The reason was the order of magnitude of the inputs: the dot product $x^\intercal.w$ would produce an array with enormous values, overflowing the sigmoid function on the second iteration. This resulted in `NaN` loss and gradient.

Since we could not change the input, we had to adjust the parameters such as the number of iterations or the learning rate. The sigmoid had already

overflowed in the second iteration of the algorithms, so the culprit could not be `max_iters`.

We then found that a significant decrease in `gamma` from 0.0005 to 1e-9 fixed the overflow issue. We observed that logistic regression rapidly converged towards the final loss value, meaning that we could decrease the number of iterations.

Setting `lambda` to 0.5 allowed even faster learning, without compromising the results.

Finally, we ended up standardizing the features, which allowed us to increase the learning rate to 0.02, and decrease the regularization term to 0.001.

4 Discussing results

We achieve an accuracy of 91.2% which is not very informative, given the class imbalance in the data. A simple always negative predictor would achieve a similar accuracy. We might obtain better results by using other more complex learning algorithms or tweaking the parameters further.